

Dart Programming

Introduction

- Client optimized programming language for apps on multiple platforms.
- Developed by Google.
- Used to build mobile, desktop, server, and web applications.
- Object oriented, class based, garbage collected language with C-style syntax.
- Can compile to either native code or JavaScript.
- Platform-independent (Windows, Mac, Linux, and Web)
 - Just-In-Time (JIT) compilation in development
 - Runs in VM;
- offering hot reload
 - Ahead-Of-Time (AOT) compilation in production
 - Native code or JavaScript; high performance

```
void main() {  
    print("Hello World!");  
}
```

void main() is the starting point where the execution of your program begins.

Every program starts with a main function.

The curly braces {} represent the beginning and the ending of a block of code.

print("Hello World!"); prints Hello World! on screen.

Each code statement must end with a semicolon.

Comments

1) `// This is a normal, Single-line comment.`

2) `/* This is`

`Multi-line comment */`

- **Variable:** Containers used to store value in the program.
- In Dart, variables can be declared using ``var``, ``final``, or ``const``.
- Variables can also be explicitly typed.

Using `var`

The `var` keyword is used when you want the compiler to infer the type of the variable based on the assigned value.

```
void main() {  
    var name = 'John Doe';    // Type inferred as String  
    var age = 30;              // Type inferred as int  
    var height = 5.9;         // Type inferred as double  
    print(name);  
    print(age);  
    print(height);  
}
```

Rules For Creating Variables

- Variable names are case sensitive, i.e., a and A are different.
- A variable name can consist of letters and alphabets.
- A variable name cannot start with a number.
- Keywords are not allowed to be used as a variable name.
- Blank spaces are not allowed in a variable name.
- Special characters are not allowed except for the underscore (_) and the dollar (\$) sign.

Data types

1. **String**: Storing text value. E.g. "John" [Must be in quotes] ' ' or " "
2. **int**: Storing integer value. E.g. 10, -10, 8555 [Decimal is not included]
3. **double**: Storing floating point values. E.g. 10.0, -10.2, 85.698 [Decimal is included]
4. **num**: Storing any type of number. E.g. 10, 20.2, -20 [both int and double]
5. **bool**: For storing true or false. E.g. true, false [Only stores true or false values]
6. **var**: For storing any value. E.g. 'Bimal', 12, 'z', true


```
void main() {  
    // declaring variables  
    String name = "John";  
    String address = "India";  
    // num age = 20; // Can take  
    // both integer and double  
    // num height = 5.9;  
    int age = 20;  
    double height = 5.9;  
    bool isMarried = false;
```

```
    // printing variables value  
    print("Name is $name");  
    print("Address is $address");  
    print("Age is $age");  
    print("Height is $height");  
    print("Married Status is  
    $isMarried");  
}
```

```
// Variable declaration with type inference  
var name = 'Hari';    // String type is inferred  
var age = 30;         // int type is inferred
```

```
// Explicit type declaration  
String country = 'India';  
int zipCode = 505001;  
double height = 5.8;  
bool isStudent = true;  
bool isVisible = true;  
bool isEnabled = false;  
dynamic dataitem = 10;
```

// Constants

```
const int DAYS_IN_WEEK = 7; // Compile-time constant
```

// Cannot be reassigned. But value can be initialized dynamically at runtime, based on some computation.

```
final birthYear = 1993;
```

// String interpolation: \${expression} to evaluate variables directly inside text.

```
print('Person: $name is $age years old');
```

```
print('Lives in: $country, $zipCode');
```

```
print('Height: ${height}ft, Student: $isStudent');
```

```
print('Birth year: $birthYear (${2026 - birthYear} years old)');
```

```
print('dataitem value is: ${dataitem}');
```

```
print(DAYS_IN_WEEK);
```

// Check data type

```
print(name.runtimeType);
```

Difference between `final` and `const`

- `final`: The value is set at runtime and can be different each time the program is run.
- `const`: The value is set at compile-time and remains constant.

Operators

- **Operands** : Represents the data.
- **Operator** : Represents how the operands will be processed to produce a value.
- Suppose the given expression is $2 + 3$.
- 2 and 3 are operands, and + is the operator.

Types Of Operators

- Arithmetic Operators
- Increment and Decrement Operators
- Assignment Operators
- Logical Operators
- Type Test Operators

Arithmetic Operators

Symbol	Operator Name	Syntax	Example	Explanation
+	Addition	$a + b$	$5 + 3$	Adds two numbers
-	Subtraction	$a - b$	$5 - 3$	Subtracts one number from another
*	Multiplication	$a * b$	$5 * 3$	Multiplies two numbers
/	Division	a / b	$5 / 3$	Divides one number by another, returns a double
~/	Integer Division	$a ~/ b$	$5 ~/ 3$	Divides one number by another, returns an integer
%	Modulus	$a \% b$	$5 \% 3$	Returns the remainder of division

```
void main() {  
    // declaring two numbers  
    int num1=10;  
    int num2=3;  
  
    // performing arithmetic calculation  
    int sum=num1+num2;    // addition  
    int diff=num1-num2;    // subtraction  
    int unaryMinus = -num1;    // unary minus  
    int mul=num1*num2;    // multiplication  
    double div=num1/num2;    // division  
    int div2 =num1~/num2;    // integer division  
    int mod=num1%num2;    // show remainder
```

```
//Printing info  
    print("The addition is $sum.");  
    print("The subtraction is $diff.");  
    print("The unary minus is $unaryMinus.");  
    print("The multiplication is $mul.");  
    print("The division is $div.");  
    print("The integer division is $div2.");  
    print("The modulus is $mod.");  
}
```


Increment and Decrement Operators

Operator Symbol	Operator Name	Description
++var	Pre Increment	Increase Value By 1. $\text{var} = \text{var} + 1$ Expression value is var+1
--var	Pre Decrement	Decrease Value By 1. $\text{var} = \text{var} - 1$ Expression value is var-1
var++	Post Increment	Increase Value By 1. $\text{var} = \text{var} + 1$ Expression value is var
var--	Post Decrement	Decrease Value By 1. $\text{var} = \text{var} - 1$ Expression value is var

```
void main() {  
    // declaring two numbers  
    int num1=0;  
    int num2=0;  
    // performing increment / decrement operator  
    // pre increment  
    num2 = ++num1;  
    print("The value of num2 is $num2");  
    // reset value to 0  
    num1 = 0;  
    num2 = 0;  
    // post increment  
    num2 = num1++;  
    print("The value of num2 is $num2");  
}
```

Assignment Operators

Symbol	Operator Name	Syntax	Example	Explanation
=	Assignment	a = b	int a = 5	Assigns a value to a variable
+=	Add and assign	a += b	a += 3	Adds and assigns a value
-=	Subtract and assign	a -= b	a -= 2	Subtracts and assigns a value
*=	Multiply and assign	a *= b	a *= 2	Multiplies and assigns a value
/=	Divide and assign	a /= b	a /= 3	Divides and assigns a value
%=	Modulus and assign	a %= b	a %= 5	Takes modulus and assigns a value

```
void main() {  
    double age = 24;  
    age+= 1; // Here age+=1 means age = age + 1.  
    print("After Addition Age is $age");  
    age-= 1; //Here age-=1 means age = age - 1.  
    print("After Subtraction Age is $age");  
    age*= 2; //Here age*=2 means age = age * 2.  
    print("After Multiplication Age is $age");  
    age/= 2; //Here age/=2 means age = age / 2.  
    print("After Division Age is $age");  
}
```

Relational Operators

Symbol	Operator Name	Syntax	Example	Explanation
==	Equality	a == b	5 == 3	Checks if two values are equal
!=	Inequality	a != b	5 != 3	Checks if two values are not equal
>	Greater than	a > b	5 > 3	Checks if one value is greater than another
<	Less than	a < b	5 < 3	Checks if one value is less than another
>=	Greater than or equal to	a >= b	5 >= 3	Checks if one value is greater than or equal to another
<=	Less than or equal to	a <= b	5 <= 3	Checks if one value is less than or equal to another

//Relational Operators

```
void main() {  
    int num1=10;  
    int num2=5;  
    //printing info  
    print(num1==num2);  
    print(num1<num2);  
    print(num1>num2);  
    print(num1<=num2);  
    print(num1>=num2);  
}
```

Logical Operators

Symbol	Operator Name	Syntax	Example	Explanation
&&	Logical AND	<code>condition1 && condition2</code>	<code>(5 > 3) && (5 < 10)</code>	Returns true if both conditions are true
 	Logical OR	<code>condition1 condition2</code>	<code>(5 > 3) (5 > 10)</code>	Returns true if at least one condition is true
!	Logical NOT	<code>!condition</code>	<code>!(5 > 3)</code>	Inverts a boolean value

```
void main(){
    int userid = 123;
    int userpin = 456;
    // Printing Info
    print((userid == 123) && (userpin== 456)); // print true
    print((userid == 1213) && (userpin== 456)); // print false.
    print((userid == 123) || (userpin== 456)); // print true.
    print((userid == 1213) || (userpin== 456)); // print true
    print((userid == 123) != (userpin== 456));//print false

}
```


Bitwise and Shift Operators

Symbol	Operator Name	Syntax	Example	Explanation
&	Bitwise AND	a & b	5 & 3	Performs a bitwise AND
	Bitwise OR	a b	5 3	Performs a bitwise OR
^	Bitwise XOR	a ^ b	5 ^ 3	Performs a bitwise XOR
~	Bitwise NOT	~a	~5	Inverts the bits
<<	Left Shift	a << n	5 << 2	Shifts bits to the left
>>	Right Shift	a >> n	5 >> 2	Shifts bits to the right

Operator	Meaning
&	AND
	OR
^	XOR
<i>~expr</i>	Unary bitwise complement (0s become 1s; 1s become 0s)
<<	Shift left
>>	Shift right
>>>	Unsigned shift right

Type Test Operators: useful for checking types at runtime.

Symbol	Operator Name	Syntax	Example	Explanation
is	Type Test	a is Type	5 is int	Checks if an object has a certain type
as	Type Cast	a as Type	a as int	Casts the object to a specified type
is!	Not Type Test	a is! Type	5 is! String	Checks if an object is not of a certain type

```
void main() {  
    String value1 = "Dart Tutorial";  
    int age = 10;  
    print(value1 is String);  
    print(age is !int);  
}
```

Conditional operator

Symbol	Operator Name	Syntax	Example	Explanation
?:	Conditional	condition ? expr1 : expr2	(5 > 3) ? 5 : 3	Returns one of two values based on a condition
??	Null-aware	a ?? b	int? a; int b = a ?? 10	Returns the left operand if not null, otherwise returns the right operand

Null Safety in Dart: Keeping it Safe

Enforces types by default, preventing null-related errors.

Use ? to declare nullable variables.

Nullish Coalescing Operator (??): Defaults Made Easy

Provides a default value if a variable is null or undefined (unlike ||).

Cleaner way to handle missing values.

Example:

String? Name;

String greeting = name ?? "There";

/ greeting will be "There" if name is null

Conditions

Types Of Condition

- **If Condition**
- **If-Else Condition**
- **If-Else-If Condition**
- **Switch case**

If Condition

```
if(condition) {  
    Statement 1;  
    Statement 2;  
    .  
    .  
    Statement n;  
}
```

```
void main()  
{  
    var age = 20  
    if(age >= 18){  
        print("You are voter.");  
    }  
}
```


If-else Condition

```
if(condition){  
    statements;  
}else{  
    statements;  
}
```

```
void main(){  
    int age = 12;  
    if(age >= 18){  
        print("You are voter.");  
    }else{  
        print("You are not voter.");  
    }  
}
```

Condition Based On Boolean Value

```
void main(){  
    bool isMarried = false;  
    if(isMarried){  
        print("You are married.");  
    }else{  
        print("You are single.");  
    }  
}
```

If-Else-If Condition

```
if(condition1){  
    statements1;  
}else if(condition2){  
    statements2;  
}else if(condition3){  
    statements3;  
}  
.  
.  
else(conditionN){  
    statementsN;  
}
```

```
// If-else statements  
int score = 85;  
if (score >= 90) {  
    print('Grade: A');  
} else if (score >= 80) {  
    print('Grade: B');  
} else if (score >= 70) {  
    print('Grade: C');  
} else {  
    print('Grade: F');  
}
```

```
void main() {  
    int noOfMonth = 5;  
  
    // Check the no of month  
    if (noOfMonth == 1) {  
        print("The month is jan");  
    } else if (noOfMonth == 2) {  
        print("The month is feb");  
    } else if (noOfMonth == 3) {  
        print("The month is march");  
    } else if (noOfMonth == 4) {  
        print("The month is april");  
    } else if (noOfMonth == 5) {  
        print("The month is may");  
    } else if (noOfMonth == 6) {  
        print("The month is june");  
    }
```

```
    } else if (noOfMonth == 7) {  
        print("The month is july");  
    } else if (noOfMonth == 8) {  
        print("The month is aug");  
    } else if (noOfMonth == 9) {  
        print("The month is sep");  
    } else if (noOfMonth == 10) {  
        print("The month is oct");  
    } else if (noOfMonth == 11) {  
        print("The month is nov");  
    } else if (noOfMonth == 12) {  
        print("The month is dec");  
    } else {  
        print("Invalid option given.");  
    }  
}
```

```
void main(){
    int num1 = 1200;
    int num2 = 1000;
    int num3 = 150;

    if(num1 > num2 && num1 > num3){
        print("Num 1 is greater: i.e $num1");
    }
    if(num2 > num1 && num2 > num3){
        print("Num2 is greater: i.e $num2");
    }
    if(num3 > num1 && num3 > num2){
        print("Num3 is greater: i.e $num3");
    }
}
```

Switch Case

```
switch(expression) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    case value3:  
        // statements  
        break;  
    default:  
        // default statements  
}
```

```
// Switch statements  
String fruit = 'Apple';  
switch (fruit) {  
    case 'Apple':  
        print('Selected fruit is red');  
        break;  
    case 'Banana':  
        print('Selected fruit is yellow');  
        break;  
    case 'Orange':  
        print('Selected fruit is orange');  
        break;  
    default: print('Unknown fruit color');  
}
```

```
void main(){
    var dayOfWeek = 5;
    if (dayOfWeek == 1) {
        print("Day is Sunday.");
    }
    else if (dayOfWeek == 2) {
        print("Day is Monday.");
    }
    else if (dayOfWeek == 3) {
        print("Day is Tuesday.");
    }
    else if (dayOfWeek == 4) {
        print("Day is Wednesday.");
```

```
    }
    else if (dayOfWeek == 5) {
        print("Day is Thursday.");
    }
    else if (dayOfWeek == 6) {
        print("Day is Friday.");
    }
    else if (dayOfWeek == 7) {
        print("Day is Saturday.");
    }else{
        print("Invalid Weekday.");
    }
}
```

```
void main() {  
    var dayOfWeek = 5;  
    switch (dayOfWeek) {  
        case 1:  
            print("Day is Sunday.");  
            break;  
        case 2:  
            print("Day is Monday.");  
            break;  
        case 3:  
            print("Day is Tuesday.");  
            break;  
        case 4:  
            print("Day is Wednesday.");  
            break;
```

```
        case 5:  
            print("Day is Thursday.");  
            break;  
        case 6:  
            print("Day is Friday.");  
            break;  
        case 7:  
            print("Day is Saturday.");  
            break;  
        default:  
            print("Invalid Weekday.");  
            break;  
    }  
}
```



```
void main() {  
    const weather = "cloudy";  
  
    switch (weather) {  
        case "sunny":  
            print("Its a sunny day. Put  
sunscreen.");  
            break;  
        case "snowy":  
            print("Get your skis.");  
            break;
```

```
        case "cloudy":  
        case "rainy":  
            print("Please bring umbrella.");  
            break;  
        default:  
            print("Sorry I am not familiar  
with such weather.");  
            break;  
    }  
}
```

Loops

- for
- While

// For loops

```
print('For loop example:');  
for (int i = 0; i < 5; i++) {  
    print(' Index: $i');  
}
```

// For-in loops (for collections)

```
print('For-in loop example:');  
for (String fruit in fruits) {  
    print(' Fruit: $fruit');  
}
```

// While loops

```
print('While loop example:');  
int counter = 0;  
while (counter < 3) {  
    print(' Counter: $counter');  
    counter++;  
}
```

Syntax:

```
do {  
/ code to be executed  
} while (condition);
```

Example: Printing numbers from 1 to 5.

```
void main() {  
int i = 1;  
do {  
print(i); i ++;  
} while (i <= 5);  
}
```

Collections : data structures that store multiple values.

- Used to hold a group of objects and provide various methods to manage these objects efficiently.

Collection Types:

- List
- Sets
- Map

Lists in Dart

1. Define List:

- A List in Dart is an ordered collection of items, where each item can be accessed by its index.
- Lists are similar to arrays . Using square brackets.

2. How to Create a List:

- Using Square Brackets:

```
List<int> numbers = [1, 2, 3, 4, 5];
```

//This creates a list of integers with elements 1, 2, 3, 4, and 5.

Using List Constructor:

```
List<String> fruits = List<String>();
```

```
fruits.addAll(['Apple', 'Banana', 'Mango']);
```

```
/* This creates an empty list of strings and then adds  
'Apple', 'Banana', and 'Mango' to it. */
```

3.Accessing List Elements:

```
print(fruits[0]); // Output: Apple  
//Accesses the first element of the list `fruits`.
```

4.Iterating Over List Elements:

```
for (var fruit in fruits) {  
    print(fruit);  
}  
/*Iterates over each element in the list and prints  
it. */
```


5.Appending Element to List:

```
fruits.add('Orange');
```

Adds 'Orange' to the end of the list `fruits`.

6.Modifying List Elements:

```
fruits[0] = 'Strawberry';
```

//Changes the first element of the list to 'Strawberry'.

7.Removing List Elements:

//Removes 'Banana' from the list `fruits`.

```
fruits.remove('Banana');
```

```
print('\n--- COLLECTIONS ---');  
// Lists (Arrays)  
List<String> fruits = ['Apple', 'Banana', 'Orange'];  
List<int> numbers = [1, 2, 3, 4, 5];  
// Accessing list elements  
print('First fruit: ${fruits[0]}');  
print('Last number: ${numbers[numbers.length - 1]}');  
// List operations  
fruits.add('Mango');  
numbers.remove(3);  
print('Fruits after modification: $fruits');  
print('Numbers after modification: $numbers');
```

Maps in Dart

1. Define Map:

- A Map is an unordered collection of key-value pairs, where each key is unique and is used to access its corresponding value.

2. Creation of Map:

- Using Curly Braces:

```
Map<String, int> scores = {'Alice': 50, 'Bob': 75};
```

//This creates a map with keys 'Alice' and 'Bob', and their corresponding values 50 and 75.

Using `Map()` Constructor:

```
Map<String, String> capitals = Map();
```

```
capitals['USA'] = 'Washington D.C.';
```

```
capitals['Canada'] = 'Ottawa';
```

//This creates an empty map and then adds key-value pairs for 'USA' and 'Canada'.

Sets

- unordered collection of unique elements. Notation: { }

```
void main() {
```

```
    Set<String> names = {'Zhang San', 'Li Si', 'Zhang San'};
```

```
    print(names); // Output: {Zhang San, Li Si}
```

```
    // Adding a duplicate element
```

```
    names.add('Li Si');
```

```
    print(names); // Output: {Zhang San, Li Si} (no change)
```

```
}
```

```
void main() {  
    Set<String> countries = {'China', 'USA'};  
    // Add an element  
    countries.add('Japan');  
    print(countries); // Output: {China, USA, Japan} (order may vary)  
    // Add multiple elements  
    countries.addAll(['France', 'Germany']);  
    print(countries); // Output: {China, USA, Japan, France, Germany}  
    // Remove an element  
    countries.remove('USA');  
    print(countries); // Output: {China, Japan, France, Germany}  
    // Clear the set  
    countries.clear();  
    print(countries); // Output: {}  
}
```

```
void main() {  
    Set<int> a = {1, 2, 3, 4};  
    Set<int> b = {3, 4, 5, 6};  
    // Union: all elements from a and b  
    Set<int> union = a.union(b);  
    print(union); // Output: {1, 2, 3, 4, 5, 6}  
    // Intersection: elements common to both a and b  
    Set<int> intersection = a.intersection(b);  
    print(intersection); // Output: {3, 4}  
    // Difference: elements in a that are not in b  
    Set<int> difference = a.difference(b);  
    print(difference); // Output: {1, 2}  
}
```

Using Map Types:

- `Map<int, String> items = {1: 'Item1', 2: 'Item2'};`
- `//Creates a map where the keys are integers and the values are strings.`

3.Accessing Map Elements:

- `print(scores['Alice']); / Output: 50`
- `//Accesses the value associated with the key 'Alice'.`

4.Iterating Over Maps Using for Loop:

```
for (var entry in capitals.entries) {  
    print('The capital of ${entry.key} is ${entry.value}');  
}
```

`//Iterates over each entry in the map and prints the key and value.`

5.Checking Whether an Element is Contained in Map:

```
print(scores.containsKey('Bob')); / Output: true
```

```
//Checks if the key 'Bob' exists in the map.
```

6.Modifying Elements in Map:

```
scores['Alice'] = 85;
```

```
//Updates the value associated with the key 'Alice' to 85.
```

7.Deleting Specific Elements:

```
scores.remove('Bob');
```

```
//Removes the key-value pair with the key 'Bob' from the map.
```

```
// Maps (Dictionaries)
```

```
Map<String, dynamic> person = {
```

```
  'name': 'Rama',
```

```
  'age': 25,
```

```
  'isEmployed': true,
```

```
  'hobbies': ['Reading', 'Playing']
```

```
};
```

```
print('Person map: $person');
```

```
print('Person name: ${person['name']}');
```

```
print('Person hobbies: ${person['hobbies']}');
```

```
// Sets (Unique collections)
```

```
Set<int> uniqueNumbers = {1, 2, 3, 3, 4, 4, 5};
```

```
// Duplicates are automatically removed
```

```
print('Unique numbers set: $uniqueNumbers');
```